Contents

Testing & Debugging Cheat Sheet **1**
Tabla de Contenidos . . . 1
1. pytest Fundamentos . . . 1
2. Fixtures . . . 3
3. Parametrización . . . 6
4. Mocking . . . 7
5. Cobertura de Tests . . . 10
6. Debugging . . . 11
7. Logging . . . 13
8. Assertions . . . 16
9. Test Patterns . . . 17
10. CLI y Configuración . . . 19
Recursos Relacionados . . . 20

Testing & Debugging Cheat Sheet

Python 3.13+ | pytest, Debugging, Logging

Tabla de Contenidos

1. pytest Fundamentos
2. Fixtures
3. Parametrización
4. Mocking
5. Cobertura de Tests
6. Debugging
7. Logging
8. Assertions
9. Test Patterns
10. CLI y Configuración

---

1. pytest Fundamentos

Estructura Básica

```python
# tests/test_calculator.py

def test_add():
    """Test básico - nombre empieza con test_"""
    assert 1 + 1 == 2

def test_subtract():
    """Cada test es independiente."""
    result = 10 - 3
    assert result == 7, "Mensaje de error personalizado"

class TestCalculator:
    """Agrupar tests en clases (opcional)."""

    def test_multiply(self):
        assert 3 * 4 == 12
```

```python
    def test_divide(self):
        assert 10 / 2 == 5
```

Ejecutar Tests

```python
# Ejecutar todos los tests
pytest

# Ejecutar archivo específico
pytest tests/test_calculator.py

# Ejecutar test específico
pytest tests/test_calculator.py::test_add
pytest tests/test_calculator.py::TestCalculator::test_multiply

# Ejecutar por keyword
pytest -k "add or subtract"

# Ejecutar con verbose
pytest -v
pytest -vv  # Más detalle

# Mostrar prints
pytest -s

# Parar en primer fallo
pytest -x
pytest --maxfail=3  # Parar después de 3 fallos

# Solo tests fallidos anteriores
pytest --lf  # last failed
pytest --ff  # failed first
```

Estructura de Proyecto

```
project/
 src/
    __init__.py
    calculator.py
 tests/
    __init__.py
    conftest.py      # Fixtures compartidos
    test_calculator.py
    test_utils.py
 pyproject.toml
 pytest.ini           # O en pyproject.toml
```

Assertions

```python
# Igualdad
assert result == expected
assert result != wrong_value

# Comparaciones
assert value > 0
```

```python
assert value >= minimum
assert value < maximum

# Identidad
assert obj is None
assert obj is not None

# Membresía
assert item in collection
assert key in dictionary

# Verdadero/Falso
assert condition
assert not condition
assert bool(value)

# Tipo
assert isinstance(obj, MyClass)
assert type(obj) is MyClass

# Aproximación (floats)
assert result == pytest.approx(3.14159, rel=1e-5)
assert result == pytest.approx(expected, abs=0.001)
```

Excepciones

```python
import pytest

def test_raises_exception():
    """Verificar que se lanza excepción."""
    with pytest.raises(ValueError):
        int("not a number")

def test_raises_with_message():
    """Verificar mensaje de excepción."""
    with pytest.raises(ValueError, match="invalid literal"):
        int("not a number")

def test_exception_info():
    """Acceder a información de la excepción."""
    with pytest.raises(ValueError) as exc_info:
        raise ValueError("custom error")

    assert "custom" in str(exc_info.value)
    assert exc_info.type is ValueError
```

---

2. Fixtures

Básico

```python
import pytest

@pytest.fixture
def sample_user():
```

```python
    """Fixture que provee datos de prueba."""
    return {"name": "Alice", "age": 30}

def test_user_name(sample_user):
    """Fixture se pasa como parámetro."""
    assert sample_user["name"] == "Alice"

def test_user_age(sample_user):
    """Cada test recibe una nueva instancia."""
    assert sample_user["age"] == 30
```

Scopes

```python
import pytest

@pytest.fixture(scope="function")  # Default - por cada test
def per_test_fixture():
    return create_resource()

@pytest.fixture(scope="class")  # Por clase de tests
def per_class_fixture():
    return create_resource()

@pytest.fixture(scope="module")  # Por archivo .py
def per_module_fixture():
    return create_resource()

@pytest.fixture(scope="session")  # Por sesión completa
def per_session_fixture():
    return create_resource()
```

Setup y Teardown

```python
import pytest

@pytest.fixture
def database():
    """Setup, yield recurso, teardown."""
    # Setup
    db = Database()
    db.connect()

    yield db  # Test recibe esto

    # Teardown (siempre se ejecuta)
    db.disconnect()

@pytest.fixture
def temp_file(tmp_path):
    """Usar otros fixtures."""
    filepath = tmp_path / "test.txt"
    filepath.write_text("test content")
    yield filepath
    # tmp_path se limpia automáticamente
```

Fixtures con Parámetros

```python
import pytest

@pytest.fixture(params=["mysql", "postgresql", "sqlite"])
def database(request):
    """Fixture parametrizado - test corre 3 veces."""
    db_type = request.param
    db = create_database(db_type)
    yield db
    db.close()

def test_insert(database):
    # Se ejecuta para cada tipo de DB
    database.insert({"key": "value"})
    assert database.count() == 1
```

Fixtures Built-in

```python
import pytest

def test_tmp_path(tmp_path):
    """Directorio temporal único por test."""
    file = tmp_path / "test.txt"
    file.write_text("hello")
    assert file.read_text() == "hello"

def test_tmp_path_factory(tmp_path_factory):
    """Factory para crear múltiples directorios."""
    dir1 = tmp_path_factory.mktemp("data1")
    dir2 = tmp_path_factory.mktemp("data2")

def test_capsys(capsys):
    """Capturar stdout/stderr."""
    print("hello")
    captured = capsys.readouterr()
    assert captured.out == "hello\n"

def test_monkeypatch(monkeypatch):
    """Modificar objetos/entorno temporalmente."""
    monkeypatch.setenv("API_KEY", "test_key")
    monkeypatch.setattr("module.function", mock_function)
    monkeypatch.delattr("module.attribute")
```

conftest.py

```python
# tests/conftest.py
# Fixtures disponibles para todos los tests del directorio

import pytest

@pytest.fixture
def api_client():
    """Fixture compartido."""
    from myapp import APIClient
    return APIClient(base_url="http://test")
```

```python
@pytest.fixture(autouse=True)
def reset_database():
    """autouse=True: se aplica a TODOS los tests."""
    yield
    Database.reset()

# Hooks
def pytest_configure(config):
    """Hook de configuración."""
    config.addinivalue_line("markers", "slow: mark test as slow")

def pytest_collection_modifyitems(items):
    """Modificar tests recolectados."""
    for item in items:
        if "slow" in item.keywords:
            item.add_marker(pytest.mark.skip(reason="Skipping slow tests"))
```

---

3. Parametrización

@pytest.mark.parametrize

```python
import pytest

@pytest.mark.parametrize("input,expected", [
    (1, 1),
    (2, 4),
    (3, 9),
    (4, 16),
])
def test_square(input, expected):
    assert input ** 2 == expected

# Múltiples parámetros
@pytest.mark.parametrize("a,b,expected", [
    (1, 2, 3),
    (0, 0, 0),
    (-1, 1, 0),
    (100, 200, 300),
])
def test_add(a, b, expected):
    assert a + b == expected
```

IDs para Tests

```python
import pytest

@pytest.mark.parametrize("input,expected", [
    pytest.param(1, 1, id="one"),
    pytest.param(2, 4, id="two"),
    pytest.param(3, 9, id="three"),
])
def test_square(input, expected):
    assert input ** 2 == expected
```

```python
# IDs generados automáticamente
@pytest.mark.parametrize("value", [
    pytest.param(0, id="zero"),
    pytest.param(-1, id="negative"),
    pytest.param(100, id="large"),
])
def test_abs(value):
    assert abs(value) >= 0
```

Parametrización Múltiple (Producto Cartesiano)

```python
import pytest

@pytest.mark.parametrize("x", [1, 2, 3])
@pytest.mark.parametrize("y", [10, 20])
def test_multiply(x, y):
    # Se ejecuta 6 veces: (1,10), (1,20), (2,10), (2,20), (3,10), (3,20)
    assert x * y == x * y
```

Skip y XFail en Parametrización

```python
import pytest
import sys

@pytest.mark.parametrize("input,expected", [
    (1, 1),
    pytest.param(2, 5, marks=pytest.mark.xfail(reason="Known bug")),
    pytest.param(3, 9, marks=pytest.mark.skip(reason="Not implemented")),
    pytest.param(
        4, 16,
        marks=pytest.mark.skipif(
            sys.platform == "win32",
            reason="Not supported on Windows"
        )
    ),
])
def test_calculation(input, expected):
    assert calculate(input) == expected
```

---

4. Mocking

unittest.mock Básico

```python
from unittest.mock import Mock, MagicMock, patch

# Mock básico
mock = Mock()
mock.method()
mock.method.assert_called_once()
mock.method.return_value = 42

# MagicMock (soporta métodos mágicos)
mock = MagicMock()
mock.__len__.return_value = 5
```

```python
len(mock)  # 5

# Configurar return_value
mock = Mock(return_value=42)
mock()  # 42

# Configurar side_effect
mock = Mock(side_effect=ValueError("error"))
mock()  # Raises ValueError

mock = Mock(side_effect=[1, 2, 3])
mock()  # 1
mock()  # 2
mock()  # 3
```

@patch Decorator

```python
from unittest.mock import patch

# Patchear función
@patch("module.function")
def test_with_mock(mock_function):
    mock_function.return_value = "mocked"
    result = module.function()
    assert result == "mocked"
    mock_function.assert_called_once()

# Patchear método de clase
@patch.object(MyClass, "method")
def test_method(mock_method):
    mock_method.return_value = 42
    obj = MyClass()
    assert obj.method() == 42

# Patchear atributo
@patch.object(MyClass, "attribute", "new_value")
def test_attribute():
    assert MyClass.attribute == "new_value"

# Múltiples patches (orden inverso de decoradores)
@patch("module.func2")
@patch("module.func1")
def test_multiple(mock_func1, mock_func2):
    pass
```

patch como Context Manager

```python
from unittest.mock import patch

def test_with_context():
    with patch("requests.get") as mock_get:
        mock_get.return_value.status_code = 200
        mock_get.return_value.json.return_value = {"data": "test"}

        response = requests.get("http://api.example.com")
```

```
        assert response.status_code == 200
        assert response.json() == {"data": "test"}
```

Assertions de Mock

```
from unittest.mock import Mock, call

mock = Mock()

# Verificar llamadas
mock(1, 2, key="value")
mock.assert_called()
mock.assert_called_once()
mock.assert_called_with(1, 2, key="value")
mock.assert_called_once_with(1, 2, key="value")

# Verificar múltiples llamadas
mock(1)
mock(2)
mock(3)
assert mock.call_count == 3
mock.assert_has_calls([call(1), call(2), call(3)])
mock.assert_has_calls([call(1), call(3)], any_order=True)

# Verificar no llamado
mock.reset_mock()
mock.assert_not_called()

# Acceder a llamadas
mock.call_args       # Última llamada
mock.call_args_list  # Todas las llamadas
```

pytest-mock (Plugin)

```
# pip install pytest-mock

def test_with_mocker(mocker):
    """mocker es un fixture de pytest-mock."""

    # Patch
    mock_func = mocker.patch("module.function")
    mock_func.return_value = "mocked"

    # Spy (llama real pero permite assertions)
    spy = mocker.spy(obj, "method")
    obj.method()
    spy.assert_called_once()

    # Stub
    mocker.stub(name="stub_name")
```

Mocking Requests HTTP

```
import pytest
from unittest.mock import patch, Mock
```

```python
@pytest.fixture
def mock_response():
    """Fixture para mock de response HTTP."""
    mock = Mock()
    mock.status_code = 200
    mock.json.return_value = {"id": 1, "name": "Test"}
    mock.text = '{"id": 1, "name": "Test"}'
    return mock

def test_api_call(mock_response):
    with patch("requests.get", return_value=mock_response):
        response = requests.get("http://api.example.com/users/1")
        assert response.status_code == 200
        assert response.json()["name"] == "Test"
```

---

5. Cobertura de Tests

pytest-cov

```bash
# Instalar
pip install pytest-cov

# Ejecutar con cobertura
pytest --cov=src
pytest --cov=src --cov-report=html
pytest --cov=src --cov-report=term-missing

# Fallar si cobertura < umbral
pytest --cov=src --cov-fail-under=80

# Múltiples directorios
pytest --cov=src --cov=lib
```

Configuración en pyproject.toml

```toml
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-v --cov=src --cov-report=term-missing"

[tool.coverage.run]
source = ["src"]
omit = [
    "*/tests/*",
    "*/__init__.py",
    "*/migrations/*",
]
branch = true

[tool.coverage.report]
exclude_lines = [
    "pragma: no cover",
    "def __repr__",
    "raise NotImplementedError",
```

```
    "if TYPE_CHECKING:",
]
fail_under = 80
show_missing = true
```

Ignorar Código

```python
def debug_function():  # pragma: no cover
    """Esta función no se incluye en cobertura."""
    pass

if __name__ == "__main__":  # pragma: no cover
    main()
```

---

6. Debugging

print() Debugging (Básico)

```python
def buggy_function(data):
    print(f"DEBUG: data = {data}")
    print(f"DEBUG: type = {type(data)}")

    result = process(data)
    print(f"DEBUG: result = {result}")

    return result

# Usar pytest -s para ver prints
# pytest -s tests/test_module.py
```

breakpoint() (Python 3.7+)

```python
def buggy_function(data):
    result = []
    for item in data:
        processed = process(item)
        breakpoint()  # Inicia debugger interactivo
        result.append(processed)
    return result

# Ejecutar: python script.py
# Comandos pdb:
# n (next)      - siguiente línea
# s (step)      - entrar en función
# c (continue)  - continuar hasta próximo breakpoint
# p variable    - imprimir variable
# l (list)      - mostrar código
# q (quit)      - salir
```

pdb (Python Debugger)

```python
import pdb
```

```python
def buggy_function():
    x = 1
    pdb.set_trace()  # Breakpoint manual
    y = x + 2
    return y

# Comandos pdb avanzados
# h (help)        - ayuda
# w (where)       - stack trace
# u (up)          - subir en stack
# d (down)        - bajar en stack
# b line          - breakpoint en línea
# b func          - breakpoint en función
# cl              - clear breakpoints
# pp expr         - pretty print
# ! statement     - ejecutar Python
```

pytest –pdb

```python
# Entrar a debugger en fallo
pytest --pdb

# Entrar en primer fallo y salir
pytest -x --pdb

# pdb en error específico
pytest --pdb-trace
```

Debugging con VS Code

```python
// .vscode/launch.json
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Python: Current File",
            "type": "python",
            "request": "launch",
            "program": "${file}",
            "console": "integratedTerminal"
        },
        {
            "name": "Python: pytest",
            "type": "python",
            "request": "launch",
            "module": "pytest",
            "args": ["-v", "-s"],
            "console": "integratedTerminal"
        }
    ]
}
```

icecream (Debugging Mejorado)

```python
# pip install icecream
from icecream import ic

def calculate(x, y):
    ic(x, y)  # ic| x: 5, y: 3
    result = x + y
    ic(result)  # ic| result: 8
    return result

# Configurar
ic.configureOutput(prefix='DEBUG | ')
ic.disable()  # Desactivar
ic.enable()   # Reactivar
```

---

7. Logging

Configuración Básica

```python
import logging

# Configuración simple
logging.basicConfig(
    level=logging.DEBUG,
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s"
)

logger = logging.getLogger(__name__)

# Usar
logger.debug("Mensaje de debug")
logger.info("Mensaje informativo")
logger.warning("Advertencia")
logger.error("Error")
logger.critical("Error crítico")
```

Niveles de Logging

Nivel	Valor	Uso
`DEBUG`	10	Información detallada para debugging
`INFO`	20	Confirmación de que todo funciona
`WARNING`	30	Algo inesperado pero no error
`ERROR`	40	Error que impide funcionalidad
`CRITICAL`	50	Error grave, programa puede fallar

Configuración Avanzada

```python
import logging
from logging.handlers import RotatingFileHandler

# Crear logger
logger = logging.getLogger("myapp")
```

```python
logger.setLevel(logging.DEBUG)

# Formatter
formatter = logging.Formatter(
    "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
)

# Console handler
console_handler = logging.StreamHandler()
console_handler.setLevel(logging.INFO)
console_handler.setFormatter(formatter)

# File handler con rotación
file_handler = RotatingFileHandler(
    "app.log",
    maxBytes=10*1024*1024,  # 10 MB
    backupCount=5
)
file_handler.setLevel(logging.DEBUG)
file_handler.setFormatter(formatter)

# Agregar handlers
logger.addHandler(console_handler)
logger.addHandler(file_handler)
```

Logging con Contexto

```python
import logging

logger = logging.getLogger(__name__)

def process_user(user_id: int):
    # Extra context
    logger.info("Processing user", extra={"user_id": user_id})

    # Formato con variables
    logger.info("User %s started task %s", user_id, task_name)

    # f-string (menos eficiente)
    logger.info(f"User {user_id} started task {task_name}")

# Logging de excepciones
try:
    risky_operation()
except Exception:
    logger.exception("Operation failed")  # Incluye traceback
    # O: logger.error("Failed", exc_info=True)
```

Configuración por Diccionario

```python
import logging.config

LOGGING_CONFIG = {
    "version": 1,
    "disable_existing_loggers": False,
```

```
    "formatters": {
        "standard": {
            "format": "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
        },
        "json": {
            "class": "pythonjsonlogger.jsonlogger.JsonFormatter",
            "format": "%(asctime)s %(name)s %(levelname)s %(message)s"
        },
    },
    "handlers": {
        "console": {
            "class": "logging.StreamHandler",
            "level": "INFO",
            "formatter": "standard",
        },
        "file": {
            "class": "logging.handlers.RotatingFileHandler",
            "level": "DEBUG",
            "formatter": "standard",
            "filename": "app.log",
            "maxBytes": 10485760,
            "backupCount": 5,
        },
    },
    "loggers": {
        "": {  # Root logger
            "handlers": ["console", "file"],
            "level": "DEBUG",
        },
        "myapp": {
            "handlers": ["console", "file"],
            "level": "DEBUG",
            "propagate": False,
        },
    },
}

logging.config.dictConfig(LOGGING_CONFIG)
```

Logging en Tests

```
import pytest
import logging

def test_logs_warning(caplog):
    """Capturar logs en tests."""
    with caplog.at_level(logging.WARNING):
        logger.warning("Test warning")

    assert "Test warning" in caplog.text
    assert len(caplog.records) == 1
    assert caplog.records[0].levelname == "WARNING"
```

---

8. Assertions

pytest Assertions Especiales

```python
import pytest

# Aproximación para floats
assert 0.1 + 0.2 == pytest.approx(0.3)
assert [0.1, 0.2] == pytest.approx([0.1, 0.2])
assert {"a": 0.1} == pytest.approx({"a": 0.1})

# Con tolerancia
assert result == pytest.approx(expected, rel=1e-3)  # Relativa
assert result == pytest.approx(expected, abs=0.01)  # Absoluta

# Verificar excepciones
with pytest.raises(ValueError):
    raise ValueError()

with pytest.raises(ValueError, match=r"invalid.*"):
    raise ValueError("invalid input")

# Verificar warnings
with pytest.warns(DeprecationWarning):
    deprecated_function()

# Skip condicional
pytest.skip("reason")
pytest.skip(allow_module_level=True)  # En nivel de módulo
```

Assertions Personalizadas

```python
def assert_valid_email(email: str) -> None:
    """Custom assertion con mensaje claro."""
    import re
    pattern = r"^[\w\.-]+@[\w\.-]+\.\w+$"
    assert re.match(pattern, email), f"Invalid email format: {email}"

def assert_dict_contains(actual: dict, expected: dict) -> None:
    """Verificar que dict contiene subset."""
    for key, value in expected.items():
        assert key in actual, f"Missing key: {key}"
        assert actual[key] == value, f"Key {key}: {actual[key]} != {value}"

# Uso
def test_user_email():
    user = get_user()
    assert_valid_email(user.email)
    assert_dict_contains(user.to_dict(), {"role": "admin"})
```

---

9. Test Patterns

Arrange-Act-Assert (AAA)

```python
def test_user_creation():
    # Arrange - preparar datos y dependencias
    name = "Alice"
    email = "alice@example.com"

    # Act - ejecutar acción a probar
    user = User.create(name=name, email=email)

    # Assert - verificar resultado
    assert user.name == name
    assert user.email == email
    assert user.id is not None
```

Given-When-Then (BDD Style)

```python
def test_user_can_add_item_to_cart():
    # Given - estado inicial
    user = create_user()
    product = create_product(price=100)
    cart = ShoppingCart(user)

    # When - acción
    cart.add(product, quantity=2)

    # Then - resultado esperado
    assert cart.item_count == 2
    assert cart.total == 200
```

Test Doubles

```python
# Dummy - objeto que se pasa pero no se usa
def test_with_dummy():
    dummy_logger = object()  # No se usa realmente
    processor = DataProcessor(logger=dummy_logger)
    assert processor.process([]) == []

# Stub - retorna valores predefinidos
def test_with_stub():
    stub_api = Mock()
    stub_api.get_user.return_value = {"name": "Alice"}

    service = UserService(api=stub_api)
    user = service.get_user(1)
    assert user["name"] == "Alice"

# Spy - registra llamadas
def test_with_spy(mocker):
    spy = mocker.spy(email_service, "send")

    user_service.register_user("alice@example.com")

    spy.assert_called_once_with(to="alice@example.com")
```

```python
# Mock - verifica comportamiento
def test_with_mock():
    mock_repo = Mock()

    service = UserService(repo=mock_repo)
    service.create_user({"name": "Alice"})

    mock_repo.save.assert_called_once()

# Fake - implementación simplificada
class FakeDatabase:
    def __init__(self):
        self.data = {}

    def save(self, key, value):
        self.data[key] = value

    def get(self, key):
        return self.data.get(key)

def test_with_fake():
    fake_db = FakeDatabase()
    service = UserService(db=fake_db)
    service.create_user({"id": 1, "name": "Alice"})
    assert fake_db.get(1) == {"id": 1, "name": "Alice"}
```

Tests de Integración

```python
import pytest

@pytest.mark.integration
class TestDatabaseIntegration:
    """Tests que requieren base de datos real."""

    @pytest.fixture(autouse=True)
    def setup_database(self, db_connection):
        """Setup y teardown por test."""
        db_connection.execute("BEGIN")
        yield
        db_connection.execute("ROLLBACK")

    def test_create_and_retrieve_user(self, db_connection):
        user_repo = UserRepository(db_connection)

        # Create
        user_id = user_repo.create({"name": "Alice"})

        # Retrieve
        user = user_repo.get(user_id)
        assert user["name"] == "Alice"
```

---

10. CLI y Configuración

Opciones de pytest CLI

```
# Básico
pytest                      # Ejecutar todos
pytest tests/               # Directorio
pytest test_file.py         # Archivo
pytest test_file.py::test_func  # Función específica

# Filtrado
pytest -k "name"            # Por nombre
pytest -m "slow"            # Por marker
pytest --ignore=tests/slow  # Ignorar directorio

# Output
pytest -v                   # Verbose
pytest -vv                  # Más verbose
pytest -q                   # Quiet
pytest -s                   # No capturar stdout
pytest --tb=short           # Traceback corto
pytest --tb=line            # Una línea por fallo

# Ejecución
pytest -x                   # Parar en primer fallo
pytest --maxfail=3          # Parar después de N fallos
pytest --lf                 # Solo últimos fallidos
pytest --ff                 # Fallidos primero
pytest -n auto              # Paralelo (pytest-xdist)

# Cobertura
pytest --cov=src
pytest --cov-report=html
pytest --cov-fail-under=80
```

Configuración en pyproject.toml

```toml
[tool.pytest.ini_options]
minversion = "7.0"
testpaths = ["tests"]
python_files = ["test_*.py", "*_test.py"]
python_functions = ["test_*"]
python_classes = ["Test*"]
addopts = [
    "-v",
    "--strict-markers",
    "--cov=src",
    "--cov-report=term-missing",
]
markers = [
    "slow: marks tests as slow (deselect with '-m \"not slow\"')",
    "integration: marks tests as integration tests",
]
filterwarnings = [
    "ignore::DeprecationWarning",
]
```

Markers Personalizados

```python
import pytest

# Definir en conftest.py o pyproject.toml
# markers = ["slow: marks tests as slow"]

@pytest.mark.slow
def test_slow_operation():
    """Test marcado como lento."""
    time.sleep(10)
    assert True

@pytest.mark.skip(reason="Not implemented yet")
def test_future_feature():
    pass

@pytest.mark.skipif(
    sys.platform == "win32",
    reason="Not supported on Windows"
)
def test_unix_only():
    pass

@pytest.mark.xfail(reason="Known bug #123")
def test_known_bug():
    assert buggy_function() == expected

@pytest.mark.parametrize("input", [1, 2, 3])
def test_parametrized(input):
    assert input > 0
```

Ejecutar por Markers

```bash
# Ejecutar solo tests lentos
pytest -m slow

# Excluir tests lentos
pytest -m "not slow"

# Combinaciones
pytest -m "slow and integration"
pytest -m "slow or integration"
pytest -m "not (slow or integration)"
```

Recursos Relacionados

- **Anterior**: File Handling Cheat Sheet
- **Siguiente**: Complete Reference
- **pytest docs**: docs.pytest.org
- **Python logging**: docs.python.org/3/library/logging.html

Cheat Sheet creado para el Bootcamp Python Zero to Hero - 2026